Health Monitoring Module: Architecture Plan

Oct 12, 2026 · @Shubham Sinha

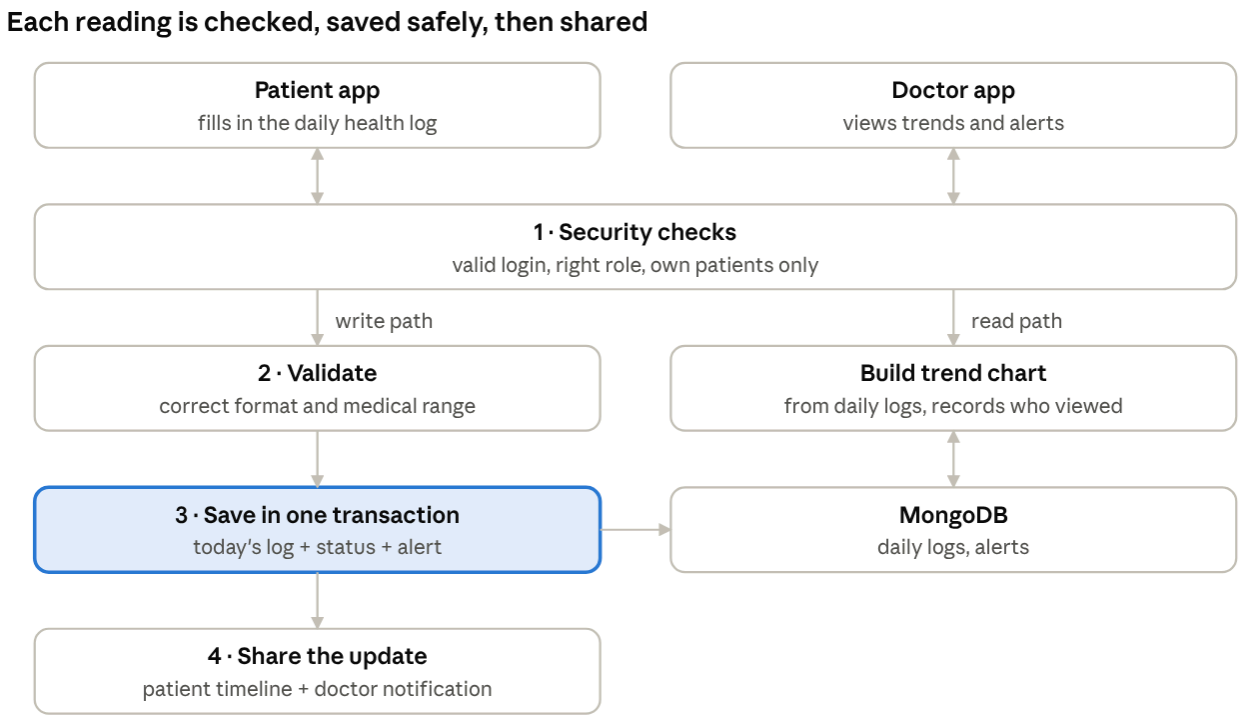
I will build the Health Monitoring backend for Fit for Surgery. Patients fill in a daily health log (blood pressure, pulse, oxygen, temperature and more), doctors see history and trends, and out-of-range readings alert the doctor. Parameters follow the SRS, and the screen flow follows the frontend design.

Why this module

- **Self-contained:** it needs only login (user ID and role) from the Auth module.
- **Used by four other modules:** the dashboard, timeline, analytics and reports all read its data.
- **Easy to demo:** a low reading entered on the phone appears as an alert on the doctor's screen.

Out of scope: device integration and risk scoring, which the SRS puts in Phase 2.

Data flow



Backend data flow · write path, read path, async events

Write path (patient fills in the daily log):

1. **Security checks:** every request needs a valid login and the right role, and doctors can reach only their own patients. The patient ID comes from the login token, never from the request.
2. **Validate:** each value sent must have the correct format and be within its medical range.
3. **Save in one transaction:** today's log is created or updated with only the fields sent, its status is recalculated (for example "9 of 14 logged"), and an alert is created if a limit is crossed.
4. **Share the update:** the patient's timeline shows "Readings updated", and the doctor is notified of any alert.

Read path (doctor views a trend): after the same security checks, the chart is built from the daily logs, one value per day, and the system records which doctor viewed which patient.

Data model (MongoDB)

The app has one form per day that patients can fill in partly and come back to later. So the module stores one document per patient per day, updated on each save.

Collection	Holds
`dailyLogs`	One document per patient per day: the readings filled in so far, the time each was logged, and the day's status
`vitalTypes`	Each parameter's unit and valid range
`alertThresholds`	Alert limits, global or per patient
`vitalAlerts`	Raised alerts, open or acknowledged
`auditLogs`	Which doctor viewed which patient, and every change to a logged value

Status, as shown in the app:

- **Each parameter:** `LOGGED` or `NOT_YET` (shown as "Not yet").
- **Each day:** `filledCount` out of `totalCount`, for example 9 of 14 (BP upper and lower count separately), and a `dayStatus` of `NOT_STARTED`, `IN_PROGRESS` or `COMPLETE`.

Parameters: all 13 from the SRS: blood pressure (upper and lower), pulse, SpO2, temperature, weight, blood sugar (optional), pain (0–10), walking duration, step count, sleep duration, nutrition intake, water intake and mood (5 levels). The current app design

shows 9 of them, so walking duration, sleep, nutrition and water intake need to be added to the form.

Main APIs

Route	Who	Does
`GET /me/vitals/today`	Patient	Today's log and status, to pre-fill the form
`PUT /me/vitals/today`	Patient	Save or update today's readings
`GET /me/vitals/trends`	Patient	Own trend charts
`GET /doctor/patients/:id/vitals/today`	Doctor	A patient's daily vitals
`GET /doctor/patients/:id/vitals/trends`	Doctor	A patient's trends
`GET /doctor/alerts`	Doctor	Open alerts
`PUT /admin/alert-thresholds/:type`	Admin	Set alert limits

Full request, response and error details are in the API reference document.

Security

- Every request needs a valid Firebase login token and the right role.
- Patients only ever reach their own data, because the patient ID comes from the token. Doctors only reach patients linked to them.
- Every database query is filtered by patient.

Design decisions

Decision	Why
One editable log per day	Matches the design: patients fill in what they can and come back later to add or change values
Status stored with the log	The app can show "Not yet" and "9 of 14 logged" without extra queries

Decision	Why
MongoDB	Readings have different shapes (BP is two numbers, mood is a level), and one daily document holds them all
One transaction per save	A log update and its alert are never saved partly
Events, not direct calls	Modules stay independent, and this one can be built and tested before teammates' modules are ready
Patient ID from the token	A patient can't reach another patient's data, even by editing a request

Plan

The module is done in about 2.5 weeks, which leaves the rest of the 4-week project for other modules.

When	Deliverable
Week 1	Schemas, saving and updating the daily log with validation and status, unit tests
Week 2	Trends, doctor views, alerts and thresholds
Week 3, first half	Connect to teammates' modules, data-isolation tests, demo data

Open questions

- [ ] Format for nutrition intake: free text or fixed options?
- [ ] Labels for the 5 mood levels (the design shows "Good" for level 4)
- [ ] Alert limits for each reading (for example, SpO2 below what %?)
- [ ] Can patients edit a previous day's log, or only today's?